table would be impractical as there are far too many game states to store.

Figure 1.14. A string diagram for a two-layer neural network. Reading from left to right, the top string diagram represents a neural network that accepts an input vector of dimension n, multiplies it by a matrix of dimensions n x m, returning a vector of dimension m. Then the nonlinear sigmoid activation function is applied to each element in the m-dimensional vector. This new vector is then fed through the same sequence of steps in layer 2, which produces the final output of the neural network, which is a k-dimensional vector.

Figure 1.15. A depiction of a Go board, an ancient Chinese game that Google DeepMind used as a testbed for its AlphaGo reinforcement learning algorithm. Professional Go player Lee Sedol only won one game out of five, marking a turning point for reinforcement learning, as Go was long thought to be impervious to the kind of algorithmic reasoning that chess is subject to. Source: http://mng.bz/DNX0.

Chapter 2. Modeling reinforcement learning problems: Markov decision processes

Figure 2.1. The standard reinforcement learning model in which an agent takes actions in an evolving environment that produces rewards to reinforce the actions of the agent.

Figure 2.2. The distribution of rewards for a simulated n-armed bandit with a 0.7 probability of payout.

Figure 2.3. This plot shows that the average reward for each slot machine play increases over time, indicating we are successfully learning how to solve the n-armed bandit problem.

Figure 2.4. With the softmax policy, the n-armed bandit algorithm tends to converge faster on the maximal mean reward.

Figure 2.5. Overview of a contextual bandit for advertisement placement. The agent (which is a neural network algorithm) receives state information (in this case, the current website the user is on), which it uses to choose which of several advertisements it should place at the